Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Final proje gösterimleri
  - Dersin tamamı dönem projelerinin canlı gösterimine ayrıldı. Her öğrenci kodunu anlattı, çalıştırdı, sıkıştırılmış dosyayı açıp kayıp olmadığını gösterdi; hoca header yapısını, hazır kod kullanımını ve sonuçları sordu.
  - Proje konuları: renk tablolu (0-255, 8 bit) görüntünün Huffman ile sıkıştırılması, JSON ve XML dosyalarının istatistiksel yöntemle sıkıştırılması, LZW ile metin sıkıştırma ve metin dosyasının yarı dinamik Huffman ile, ikili karakter grupları (bigram) da kodlanarak sıkıştırılması.
  - Görüntü projesinin metni: "Verilen bir resim dosyasını (0-255 arası renklerden oluşan, Color Table) Huffman kodlama algoritmasını kullanarak sıkıştırınız. Sıkıştırdığınız dosyayı açan kodu da yazmalısınız. İşlem sonucunda bir kayıp olmadığını gösteriniz." (ekrandan, 02:10:00)
- Huffman ile görüntü sıkıştırma: ortak akış
  - Piksel değerlerinin histogramı (frekans tablosu) çıkarılır, frekansa göre sıralanır, Huffman ağacı kurulur, ağaç dolaşılarak her piksel değerine bir bit dizisi verilir.
  - Görüntü tek boyutlu diziye indirgenir, her piksel kodla değiştirilip uzun bir bit dizisi elde edilir; bit dizisi 8'erli bölünüp bayt olarak yazılır. Sondaki eksik bitler sıfırla doldurulur (padding), kaç bit eklendiği saklanır.
  - Açma tarafı aynı ağacı ya da kod tablosunu header'dan alır, bit dizisini çözer, diziyi görüntü boyutlarına göre yeniden matrise çevirir.
  - Kayıpsızlık, orijinal ve açılmış piksel matrislerinin farkının sıfır olmasıyla ya da dosyaların bayt bayt karşılaştırılmasıyla gösterildi.
- Kuantalama ile birlikte Huffman
  - Bir projede Huffman'dan önce K-Means ile renk sayısı 32'ye (ya da 64, 128'e) indirildi. Kuantalamasız denemede en kısa kod 7 bit, 32 renkte 5 bit oldu.
  - Sonuçlar (ekrandan, 00:13:56): orijinal sıkıştırma oranı %79,07; kuantalanmış görüntünün kendine göre %83,00, orijinale göre %82,94.
  - Başka bir projede 16 renge kuantalama kazancı %3'ten %50'ye çıkardı, 64 renkte %26 oldu; kuantalamasız Lena'da kazanç yaklaşık %6 kaldı (derste sözlü, 01:32:10).
  - Bir projede kuantalama, değeri en yakın 10'un katına yuvarlayarak yapıldı (12 → 10, 18 → 20, 161 → 160); 66.614 bayt → 34.365 bayt, oran 0,5159, kazanç %48,41 (ekrandan, 01:50:25).
- Header tasarımları
  - Kod tablosu (piksel değeri ve kodu) ya da ağacın kendisi gönderildi. Bir projede iki header türü karşılaştırıldı: ağacı göndermek açmada daha yavaştır ama daha çok sıkıştırır. Ağaçlı header ile 41.078 bayt → 34.795 bayt, oran 0,8470, kazanç %15,30 (ekrandan, 00:33:50).
  - Ayrıntılı header örneği (ekrandan, 01:30:00): genişlik ve yüksekliğin 255'e bölümü ve kalanı (her biri 8 bit), header uzunluğunun 255'e bölümü ve kalanı (8'er bit), kanal sayısı (3 bit), Huffman ağacının LNR (sol-düğüm-sağ) bit dizisi, renk sayısı (9 bit), renkler (renk sayısı × 8 bit).
  - Metin projesinde dosya yapısı (ekrandan, 01:53:30): 3 bit padding boyu, 32 bit ağaç uzunluğu, n bit ağaç, m bit kodlanmış metin, $8-((3+32+n+m) \bmod 8)$ bit padding.
- Sonuçların görüntüye bağımlılığı
  - Az renkli, düz bölgeli görüntüler çok sıkışır: tek bir değerin (ör. 131) baskın olduğu

8 bitlik bir görüntüde bu değer tek bitlik `0` kodunu aldı, ortalama kod uzunluğu 1,31 bit, kazanç %83 (ekrandan, 00:48:50).
  - Renkli bir BMP'de oran 0,210931, kazanç %78,906921 (ekrandan, 00:45:14); başka bir görüntüde en kısa kod 1, en uzun 14, ortalama 7,5 bit, kazanç %61,94 (ekrandan, 01:31:40).
  - Çok renkli fotoğraflarda kazanç düşüktür: örneğin 1.784.160 bit → 1.611.724 bit, %9,664 (ekrandan, 00:40:00).
- JSON ve XML sıkıştırma
  - Bir proje aritmetik kodlama kullandı: 256 bayt değeri ve bir EOF sembolü için 257 elemanlı frekans tablosu, kümülatif frekanslarla low-high güncellemesi (ekrandan, 00:20:00). Bir JSON dosyası 25.529 KB → 16.581 KB, kayıpsız.
  - Diğer proje Shannon-Fano kullandı; header'da sözlükteki karakter sayısı, her karakter ve kod uzunluğu ile kod içeriği ayırıcı (`\x00`) ile yazıldı (ekrandan, 01:20:30). Büyük JSON'da 652.309 → 347.775 bayt, oran 0,4669 (yaklaşık %53 küçülme) (ekrandan, 01:20:00).
  - Her iki proje de karakter tabanlıydı; hoca, etiket ve kalıp tabanlı kodlamanın çok daha iyi sonuç vereceğini söyledi.
- LZW ile metin sıkıştırma
  - Sözlük önce tek karakterlerle başlatılır, yeni girdiler arkasından eklenir. Bir projede sözlük 200 elemanla başlatıldı, yeni dizgiler 200, 201, 202, … kodlarını aldı (ekrandan, 01:15:05).
  - Küçük dosyalarda ve tekrar içermeyen verilerde (rastgele kelimeler, bir milyon basamaklı pi sayısı) dosya büyür. Doğal dil metinlerinde yaklaşık %65-75 kazanç alındı, özel hazırlanmış tekrarlı dosyalarda %98'e çıktı.
  - Sözlük boyutunu sınırsız bırakmak ile sınırlamak karşılaştırıldı; sınırlı sözlükte performans biraz düştü.
- Yarı dinamik Huffman ve bigramlar
  - Metindeki tek karakterlerin yanında en sık geçen ikili grupların bir eşik oranı kadarı (ör. 0,2 ile en sık %20'si) da sembol yapıldı.
  - Bir Shakespeare metninde 125.179 bayt → 72.806 bayt, kazanç %41,84; boşluk 19.359 kez geçip `0001`, `e` 10.380 kez geçip `1111` kodunu aldı; `th`, `he`, `an`, `er` gibi ikililer de kod aldı (ekrandan, 01:55:05).

Hocanın Özellikle Vurguladığı Kısımlar

- JSON, XML ve HTML'de kalıp tabanlı kodlama
  - Bu formatlarda sürekli tekrar eden etiket ve kalıplar vardır. Karakter karakter kodlamak yerine her kalıba (ör. bir bitiş etiketine ya da HTML'deki bir referans kalıbına) tek bir kod verilirse çok daha fazla sıkıştırma elde edilir. Bigram kodlayan metin projesi bu fikrin küçük bir örneğidir.
- Açan taraf tabloyu bilmek zorundadır
  - Aritmetik kodlamada da Huffman'da da karşı taraf sembol frekanslarını ya da kodları bilmeden açamaz; bu bilgi header'da gönderilmelidir. Hoca her gösterimde header'da ne gönderildiğini sordu.
- Kuantalama olmadan görüntüde yüksek kazanç alınamaz
  - Çok renkli görüntülerde renk sayısı kuantalamayla azaltılmazsa Huffman'ın kazancı düşük kalır. Kuantalama isteğe bağlıdır ve kayıplıdır; kuantalanmış görüntüden orijinale dönülemez.
- Kayıpsızlığı göstermek
  - Açılan dosyanın orijinalle aynı olduğu ekranda ya da bayt düzeyinde gösterilmelidir. Hoca, açılan görüntüyü çizdirmeyen bir öğrenciye BMP header'ını saklayıp resmi yeniden çizdirmeyi önerdi.
- Doğal dil metinlerinde beklenen kazanç
  - Standart doğal dil metninde yaklaşık %75 kazanç beklenir, belki %80'e çıkar. Harf serilerinden oluşan özel dosyalar gerçek hayatta yoktur. Ne kadar uzun kalıp yakalanırsa o kadar başarılı olunur.

Kısa Tekrar Notları

- Huffman görüntü sıkıştırma: histogram → sıralama → ağaç → kodlar → bit dizisi → 8'li baytlar + padding.
- En sık piksel değeri en kısa kodu alır; tek bir değer baskınsa kod uzunluğu 1 bit olabilir.
- Header seçenekleri: kod tablosu, frekanslar ya da ağacın kendisi; açma tarafı aynı ağacı kurmalıdır.
- Sıkıştırma yüzdesi $= (1 - \text{çıktı}/\text{girdi}) \times 100$; oran (ratio) $= \text{çıktı}/\text{girdi}$.
- Kuantalama renk sayısını azaltıp kazancı artırır, ama kayıplıdır.
- JSON/XML/HTML: karakter yerine kalıplara kod vermek gerekir.
- LZW küçük dosyada ve tekrarsız veride dosyayı büyütebilir; uzun tekrarlarda çok başarılıdır.
- Yarı dinamik Huffman'da sık bigramlar da sembol yapılabilir; eşik değeri hangi bigramların alınacağını belirler.

Detaylı Açıklamalar

- Renk tablolu (palette) görüntülerde Huffman
  - 8 bitlik renk tablolu bir görüntüde her piksel 0-255 arasında bir indistir; asıl renk (kırmızı, yeşil, mavi değerleri) renk tablosundadır. Huffman yalnız indisleri kodlar; açılan görüntünün renkli görünmesi için renk tablosunun da korunması gerekir. Birkaç öğrenci, piksel matrisleri ve dosya boyutları birebir aynı olduğu halde açılan görüntünün gri çıktığını gösterdi. Bir öğrenci sorunun BMP header'ı ile ilgili olduğunu, matristen yeni bir görüntü oluşturulunca varsayılan olarak gri seviye kabul edildiğini söyledi; bir başkası paleti ve öznitelikleri kopyalamanın işe yaramadığını, ancak orijinal görüntü nesnesini doğrudan kopyalayınca kaydedebildiğini belirtti.
- Header'ı küçültmek
  - Bir öğrenci kod tablosunu gönderen ve ağacı gönderen iki header türünü karşılaştırdı. Ağaç gönderilince header küçülür, sıkıştırma artar (ağaçlı header'da oran 0,847 ve kazanç yaklaşık %15, diğerinde oran yaklaşık 0,87 ve kazanç yaklaşık %12), ama açma tarafı ağacı yeniden kurmak zorunda olduğu için daha yavaştır.
  - Başka bir öğrenci büyük sayıları (genişlik, yükseklik, header uzunluğu) 255'e bölüm ve kalan olarak iki baytta yazdı, renk sayısını 9 bitle, kanal sayısını 3 bitle gösterdi; 2, 3 ve 4 kanallı görüntülerde denedi.
- Aritmetik kodlamada header
  - JSON/XML projesinde her baytın frekansı sayıldı; 257. sembol dosya sonunu (EOF) gösterir. Kod ekranında açma fonksiyonu 256 frekansı dosyanın başından `readInt(in, NUM_BIT)` ile okuyor, `freq[256] = 1` ile EOF frekansını ekliyor (ekrandan, 00:20:00); yani frekans tablosu sıkıştırılmış dosyanın başına yazılmıştır.
- LZW sözlük boyutu
  - Sözlük açılan boyutun tamamı karşı tarafa taşınmamalıdır; hoca, sözlüğün yalnız dolu kısmının kullanılmasını önerdi. Sözlüğü 256 yerine daha büyük başlatmak Türkçe karakterler için düşünülmüştü; küçük metinlerde bu, kazancı düşürdü.
  - Bir öğrenci kodları ondalık metin olarak yazdığından bir karakter üç karakterle gösterildi ve küçük dosyalar büyüdü.
- Kuantalama yöntemi
  - Değerleri en yakın 10'un katına yuvarlamak, aralığı eşit parçalara bölüp her değeri bir parçanın sınırına taşımaktır. Hoca bu yöntemin bir adı olduğunu, aralığın parçalara bölünüp alt ve üst sınırlara yuvarlandığını söyledi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.